Faculty of Mathematics
and Computer Science

Tehnici de realizare a sistemelor inteligente (TRSI 2026)

Detection of Network Fuzzing Traffic using Machine Learning

Farcaș Andrei-Cătălin

Department of Computer Science, Babeş-Bolyai University
1, M. Kogalniceanu Street, 400084, Cluj-Napoca, Romania
E-mail: andreifarcas21@gmail.com

Abstract

The rapid development of sophisticated cyber threats has created a constant demand for more effective Network Intrusion Detection Systems (NIDS). While many studies explore machine learning (ML) for next-generation NIDS, they often treat all attack types equally, leading to high false-positive rates in practical scenarios. This paper specifically focuses on network fuzzing, an evolving threat driven by advanced vulnerability discovery tools such as Peach Fuzzer, SPIKE, and AFL. This investigation provides a comparative analysis of several supervised learning algorithms, including Random Forest, Decision Trees, and Support Vector Machines, tested specifically on their ability to detect fuzzing traffic. By synthesizing performance results from recent literature using modern datasets like UNSW-NB15 and Kitsune, this study evaluates the results of each model. After a complete analysis, [X Algorithm] offers the most robust performance, making it a highly recommended candidate for improving the accuracy and reliability of future Network Intrusion Detection Systems.

Keywords:
Machine Learning; Network Fuzzing; Intrusion Detection System; Network Security; Comparative Analysis; UNSW-NB15 Dataset; Kitsune Network Attack Dataset

1. Introduction

Network security represents a critical aspect of global connectivity, given the ever-increasing number of devices actively participating in data transmission. In this context, data safety plays an essential role for every participant in the Internet infrastructure. To provide a solution for attack prevention and detection, the concept of a Network Intrusion Detection System (IDS) was developed, which serves to detect anomalies occurring in network traffic as a layer above the defense levels represented by software tools such as Firewalls, Antivirus, Antimalware, or Antispyware [15]. In some cases, these systems prove insufficient against modern vulnerability discovery techniques, particularly Network

Fuzzing techniques, which generate malformed traffic to expose weaknesses in protocol implementations. Consequently, in a world increasingly dependent on digital infrastructures, the need for innovation in anomaly detection and the identification of malicious network traffic is constantly growing.

Among the various methods for detecting vulnerabilities in the context of cybersecurity, Network Fuzzing is a technique increasingly used and studied by researchers to discover and address vulnerabilities in network protocols and their implementations. This technique involves transmitting a high volume of malformed packets at various layers of the OSI model to test the robustness of protocol implementations on target devices. The method is used for both offensive and defensive purposes, as there are numerous situations where classical defense systems can be bypassed by this type of traffic, precisely because the generated packets do not follow patterns known to these systems [9]. Therefore, the academic community has focused its attention on this issue, documenting it and developing relevant datasets for the analysis of such malicious network traffic [11].

With the evolution of attack techniques, the scientific community has explored the use of Machine Learning methods as an adaptive solution for anomaly detection problems in network traffic. Unlike classical systems based on static rules, supervised classification models can learn complex patterns directly from data without requiring an explicit definition for each individual type of attack [13]. This approach involves training a model on labeled datasets, where each record represents a network flow classified as normal or malicious, thereby allowing the model to generalize and identify previously unseen traffic. In the literature, algorithms such as Decision Trees, Random Forests, and Logistic Regression have been successfully applied in the context of IDS systems, demonstrating that supervised classification methods represent a viable and active research direction in the field of network security [7].

An essential aspect in the development of classification models for IDS systems is the choice of the dataset used for training and evaluation. In the literature, the most frequently used historical dataset was KDD CUP 99, whose fundamental limitations have been documented since 2009 [14], prompting the academic community to seek alternatives more representative of modern network traffic. In response to these limitations, more recent datasets have been developed, among which UNSW-NB15 stands out for the diversity of included attack categories, covering types of malicious traffic such as Fuzzers, Exploits, DoS, Backdoors, and Shellcode, generated under controlled conditions with real tools [11]. The choice of the dataset significantly influences the results obtained, and existing studies demonstrate that models trained on more recent and diverse data tend to generalize better in the context of modern threats [13].

This report aims to synthesize existing approaches in the literature regarding the use of supervised classification methods for anomaly detection in network traffic, with an emphasis on Network Fuzzing techniques. A comparative analysis of results obtained in current research for machine learning algorithms will be conducted, highlighting the strengths and limitations of each approach within the context of Intrusion Detection Systems (IDS).

2. Practical importance of IDS and Network Fuzzing

Throughout history, researchers tried to develop a system to ensure the security of networked communications, so the first attempt at such a system was done by Dorothy E. Denning, who published an academic paper titled ”An Intrusion Detection Model” [4]. Following this paper, the Stanford Research Institute (SRI) developed the Intrusion Detection Expert System (IDES), which used statistical anomaly detection and signature profiles to detect malicious network behaviors. During the 1990’s, firewalls were very effective, but IDS systems remained the most secure practice due to the evolving new threats [15].

These systems changed their paradigm when Gosh and Schwartzbard published an academic paper in 1999 [6] where they proposed a different approach to how Intrusion Detection Systems work, introducing the use of machine learning methods. The authors have proven that ML models are capable of identifying patterns in the network traffic and therefore are capable of surpassing the previous models. However, the study also highlighted a major problem in this approach, which is the significant number of false positive cases.

Machine learning-based Intrusion Detection Systems use mathematical algorithms to analyze network traffic and effectively distinguish intrusions from normal activities. These procedures typically involve a training phase during which the learning model is exposed to datasets containing labeled samples of both attack and legitimate traffic classes. In supervised learning, the model is provided with input data and the corresponding outputs, which results

in predicting future data behavior. Following this training phase, the model is tested on separate, independent data samples to verify the accuracy of its predictions [5].

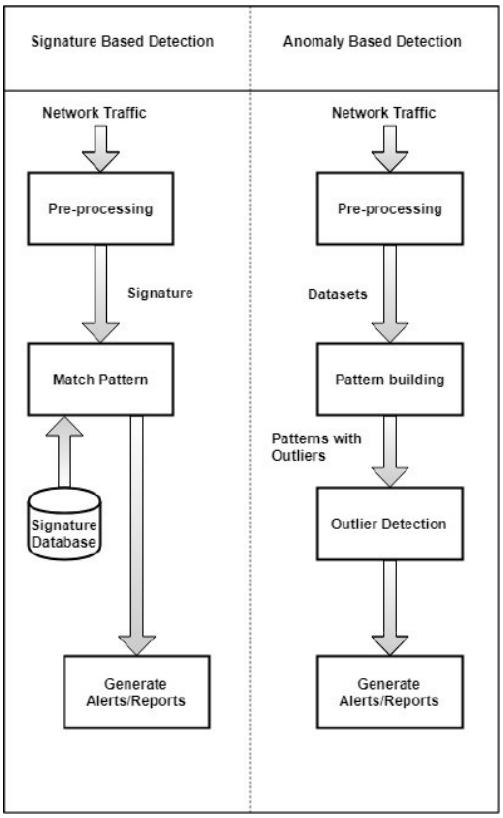

Fig. 1. Signature and Anomaly IDS architecture. Source: [5]

As Intrusion Detection Systems (IDS) became a standard part of network defense, researchers began looking for ways to handle complex threats like fuzzing. For a long time, fuzzing was not treated as a separate category in detection systems, partly because historical datasets like KDD CUP 99 [14] did not include it as a specific class for training. This changed with the creation of the UNSW-NB15 dataset [11]. This was one of the first major datasets to include ”Fuzzers” as a distinct attack category among its nine classes of malicious traffic. The data was generated using real tools in a controlled environment, allowing models to finally learn the unique patterns of malformed packets. As network technology evolved, even more complex data was needed, so the Kitsune Network Attack Dataset [10] represents the next step in this evolution. This dataset provided a much larger and more diverse set of fuzzing attacks, specifically focused on modern environments like IoT networks. By including a wider variety of protocol violations and malformed inputs, Kitsune allowed researchers to train models that are much more robust.

Even though fuzzing is an important attack type, it is not often studied in IDS research. Most researchers focus on general categories like DoS or Probes instead of looking specifically at fuzzing performance. However, fuzzing traffic is different from other attacks because it has unique features. Detecting these attacks effectively may require special classification approaches that are different from those used for general intrusion detection.

3. Machine Learning Approaches for Network Fuzzing Detection

The methodology of this comparative analysis focuses on extracting performance metrics specifically for the Network Fuzzing category. While most academic papers address Network Intrusion Detection as a general problem, this study filters the results to highlight how algorithms handle malformed packets. We analyzed research papers that uti-

lize multi-class classification on the UNSW-NB15 and Kitsune datasets, as these provide specific labels for fuzzing traffic. By isolating the F1-Score and Detection Rate for the 'Fuzzers' class, we can provide a more accurate evaluation of model robustness against unpredictable traffic patterns. Each algorithm will be presented in its section, and after the analysis, a table with the conclusions will be presented.

3.1. Linear and Statistical Methods

Linear and Statistical methods represent the most computationally efficient approaches for network traffic intrusion detection, because they offer high interoperability at the cost of reduced capacity to model complex, non-linear relationships in network traffic data. In the context of network fuzzing traffic detection, two representative methods for this paradigm have been evaluated in literature: Logistic Regression (LR) and Linear Discriminant Analysis (LDA).

In the paper written by Abuzir and Abu Khalil [1], they have evaluated both methods on the Kitsune Fuzzing Dataset, which is a large-scale network traffic dataset containing approximately 2.24 million packet records captured from a surveillance system environment, in which a web API fuzzing attack was initiated after approximately one million packets of normal traffic. The dataset was preprocessed using the AfterImage feature extractor, producing 115 statistical features per packet, and split into training and testing sets using a standard train-test split procedure. After that, the outlier detection was performed using the Interquartile Range (IQR) method, and feature selection was conducted through correlation analysis.

Logistic Regression demonstrated strong overall performance on this dataset, achieving an accuracy of 0.79 on the raw test set before the outlier removal, with the cleaned dataset producing improved results. Following preprocessing, the model achieved perfect classification scores across both benign and malicious classes, with precision, recall, and F1-score all reaching 1.00 for the malicious class. These results suggest that, on preprocessed and balanced fuzzing traffic data, Logistic Regression can serve as a reliable and computationally inexpensive baseline classifier. The authors attribute this performance to the effectiveness of the preprocessing pipeline, which reduced noise and improved the linear separability of the feature space.

In contrast, Linear Discriminant Analysis showed a different behavior on the same dataset. While achieving high precision of 0.90 for the malicious class, the model's recall dropped to 0.24, resulting in an F1-score of only 0.38 for attack detection. This significant difference from the previous model indicates that LDA correctly identified only approximately one in four actual fuzzing attack cases, making it unreliable for intrusion detection purposes despite its high overall accuracy. The authors note that this failure is the result of the class imbalance present in the dataset, where normal traffic records outnumber attack records, causing LDA to bias its decision boundary towards the majority class. This finding highlights a fundamental limitation of linear discriminant approaches in the context of anomaly detection on imbalanced network traffic datasets, where the cost of false negatives is considerably higher than the cost of false positives.

3.2. Tree-Based Methods

Decision tree-based classifiers are a popular choice for network intrusion detection because they are fast and easy to understand. These models work by creating a hierarchy of simple rules to classify network traffic as normal or malicious. In recent studies focused on network fuzzing, tree-based methods have shown strong performance, but their success often depends on how the training data is prepared and which specific version of the algorithm is used.

Research by Alabdulatif and Rizvi [2] tested five different versions of Decision Trees - Fine, Medium, Coarse, Boosted, and Bagged - using the Kitsune Fuzzing Dataset. This dataset is massive, covering 115 input features. Most of these models, including the Fine, Medium, Coarse, Bagged, and RUS Boosted Trees, performed almost perfectly, hitting 99.99% accuracy and a 100% True Positive Rate (TPR). The Fine Tree stood out for efficiency, as it only cost 363 seconds to train and could process 890,000 observations per second. While the Bagged Tree managed to get zero misclassifications, it was much heavier on the system, taking 1,274 seconds to train, making it 3.5 times slower than the Fine Tree.

Interestingly, the Boosted Tree struggled with fuzzing data. While it usually performs well on other tasks, its TPR dropped to just 80%, and its "misclassification cost" was thousands of times higher than the other models. The researchers believe this happened because fuzzing traffic is inherently "adversarial." It introduces irregular, chaotic patterns that mess with the way boosting builds models sequentially, leading to a significant amount of errors. This

is an important aspect to keep in mind for anyone building an IDS: boosting strategies might need extra help or a different setup if they're going to catch fuzzing attacks reliably.

In the world of car security, Alalwany et al. [3] looked at Decision Trees alongside Random Forest and XGBoost using the CAN Bus Intrusion Dataset [8]. This real-world dataset from 2020 includes over 3.6 million messages, with about 90,000 of them being fuzzing attacks. To get the best results, they used a technique called "Near-Miss" to balance the data and "Extra Trees" to pick out the most important features. After tuning the settings—like setting a maximum depth of 20, the Decision Tree model was nearly perfect, with Precision, Recall, and F1-scores all at or above 0.99. This proves that with the right preparation work (like balancing and feature selection), Decision Trees are incredibly effective at spotting fuzzing in specialized environments like a vehicle's network.

3.3. Ensemble Methods

Ensemble learning methods improve the performance of a system by combining multiple models to get more accurate and reliable predictions. This approach helps reduce errors and prevents the model from simply memorizing the training data, a common problem for single decision trees. In the study of network fuzzing, Random Forest is the most widely analyzed ensemble method, although other strategies like stacking, bagging, and voting are also frequently discussed in the literature.

Alalwany et al. [3] provide the most detailed per-class results for fuzzing traffic detection using ensemble methods, evaluating Random Forest, Stacking, Bagging, and Voting on the CAN Bus Intrusion Dataset, a real-world automotive network dataset, where 89,879 (2.45%) entries were labelled as fuzzing attacks. Following data balancing using the Near-Miss undersampling technique and feature selection using the Extra Trees method, Random Forest achieved precision of 0.99, recall of 1.00, and F1-score of 1.00 on the fuzzing class, demonstrating the algorithm's capacity to identify all fuzzing attack instances while maintaining minimal false positives. The Stacking ensemble, which combined Random Forest, Decision Tree, and XGBoost as base learners with XGBoost as a meta-learner, achieved identical perfect scores on the fuzzing class while also attaining the highest overall accuracy of 0.986 across all attack types. Bagging and Voting achieved slightly lower overall performance, with F1-scores of 0.980 and 0.981 respectively, though both maintained strong per-class results on fuzzing detection. The authors further demonstrate that incorporating feature selection consistently improved precision, recall, and F1-score across all ensemble methods, while simultaneously reducing training and testing times. The exception being Stacking, which incurred additional training overhead due to its two-stage architecture.

Shushlevska et al. [13] also evaluated Random Forest on the UNSW-NB15 dataset alongside Naive Bayes, Logistic Regression, and Decision Tree, using 5-fold cross-validation on 257,673 records with 49 features across nine attack categories, including Fuzzers. Random Forest achieved the highest performance among all evaluated models, with a CV F1-score mean of 0.971, a test Recall of 0.985, and an AUC of 0.977. However, these results represent aggregate performance across all nine attack categories combined, and no per-class breakdown for the Fuzzers category is reported. While this study confirms the general superiority of Random Forest over other classifiers on UNSW-NB15, it cannot be used to draw direct conclusions about its effectiveness specifically on fuzzing traffic, which is precisely the gap in the literature that motivates the focus of this report.

Across both studies, Random Forest proves to be the most consistently reliable ensemble method for network intrusion detection in the presence of fuzzing traffic. The results from Alalwany et al. additionally suggest that more advanced strategies such as Stacking can further improve performance, though the marginal gains over Random Forest may not justify the increased training complexity in resource-constrained deployment environments.

3.4. Comparative Analysis

Table 1. Comparative Performance Synthesis of ML Algorithms on Fuzzing Class.

Model	Author	Dataset	Prec.	Rec.	F1	Acc.	Time	Size	Observation
LR	Abuzir 2025	Kitsune	1.00	1.00	1.00	1.00	N/A	2.24M	Optimized via IQR.
LDA	Abuzir 2025	Kitsune	0.90	0.24	0.38	0.89	N/A	2.24M	High bias; low recall.
Fine Tree	Alabd. 2022	Kitsune	1.00	1.00	1.00	99.99%	363s	1.75M	High efficiency.
Bagged Tree	Alabd. 2022	Kitsune	1.00	1.00	1.00	99.99%	1297s	1.75M	Zero misclass.
Boosted Tree	Alabd. 2022	Kitsune	0.80	1.00	0.88	80.71%	451s	1.75M	Perf. collapse.
RF (tuned)	Alalwany 2025	CAN Bus	0.99	1.00	1.00	0.978	21.6s	3.67M	IVN specialized.
DT (tuned)	Alalwany 2025	CAN Bus	1.00	0.99	0.99	0.971	1.5s	3.67M	High speed.
Stacking	Alalwany 2025	CAN Bus	1.00	1.00	1.00	0.986	114s	3.67M	Top accuracy.
XGBoost	Alalwany 2025	CAN Bus	1.00	1.00	1.00	0.985	6.4s	3.67M	Best efficiency.
RF (Aggr.)	Shushl. 2022	NB15	0.817	0.985	0.893	87.09%	59.1s	257k	Not conclusive.
NB (Aggr.)	Shushl. 2022	NB15	0.687	0.853	0.761	70.62%	0.37s	257k	Not conclusive.

Notes on data sources:

- **Abuzir & Abu Khalil (2025) [1]**:
  - LR and LDA: performance metrics for the *Malicious* class from Table 6 (p. 17).
  - Kitsune Fuzzing dataset size: 2,244,139 records (p. 9, df.info() output).
- **Alabdulatif & Rizvi (2022) [2]**:
  - Tree models (Fine, Bagged, Boosted): Table 3, *Performance matrix on fuzzing* (journal p. 833; PDF p. 7).
  - Training time: same table, *Train time* column.
  - Train/test split: 70% training (1,752,987), 30% testing (751,280) – Section 5 (p. 831).
- **Alalwany et al. (2025) [3]**:
  - Per-class metrics for *Fuzzing*: Table 7 (p. 15).
  - Mean accuracy (10-fold CV): Table 5 (p. 13).
  - Training time with feature selection: Figure 9 (p. 16).
- **Shushlevska et al. (2022) [13]**:
  - Raw performance (RF, NB): Table 2 (p. 5).
  - Classification over 10 categories (9 attacks + normal): Section 3 (p. 3).

The results given in Table 1. show that the XGBoost algorithm is the most efficient algorithm for detecting fuzzing attacks, offering an optimal balance between predictive performance and computational overhead. While several models achieve near-perfect classification metrics, XGBoost distinguishes itself through its exceptional training efficiency. Specifically, it scores perfect precision, recall, and F1-score (1.00 each) with an accuracy of 0.985, while requiring only 6.4 seconds of training time.

Among the competing algorithms, Stacking achieves comparable predictive performance (F1 = 1.00, accuracy = 0.986) but demands approximately 18 times longer training (114 seconds), making it less suitable for rapid model iteration or resource-constrained environments. Fine Tree also performs perfect detection metrics (F1 = 1.00, accuracy = 99.99%) yet requires 363 seconds of training — nearly 57 times slower than XGBoost. Moreover, the tuned Decision Tree (DT) offers the fastest training time (1.5 seconds) but suffers from a recall of 0.99, meaning it would miss 1% of actual fuzzing attacks in practice. Random Forest (tuned) presents a similar trade-off with a precision of 0.99 (1% false positives) and training time of 21.6 seconds.

4. Datasets used in IDS research

Another important aspect to talk about are the dataset on which these models are trained, since the data needs to be as close to reality as possible and very accurate in relation to the specific attacks. The pioneer of network intrusion detection dataset for training NIDS Machine Learning models is KDD Cup 1999 [14], which was, and still is, widely used as one of the few publicly available datasets for network-based anomaly detection systems. The problem of this dataset, in regard with this paper's main focus, is that it lacks the attack type of network fuzzing. To bridge this gap, several next-generation datasets were introduced: UNSW-NB15 (2015) [11] provided a more diverse range of

modern attacks; CICIDS2017 (2017) [12] offered a comprehensive profile of contemporary network protocols; and Kitsune (2018) [10] specialized in anomaly detection for IoT and high-speed traffic. These datasets represent a crucial evolution, as they were specifically designed to incorporate fuzzing as a primary attack category.

A recurring obstacle in training machine learning algorithms on network intrusion is class imbalance. A fundamental characteristic of these datasets is the fact that most of the publicly available ones have a significant proportion of the traffic as normal, far exceeding that of attack types, including fuzzing. For example, the Kitsune dataset [10], which has the most amount of fuzzed network traffic, has 2.24 million records, but exact malicious attacks are lower then normal traffic. Moreover, the UNSW-NB15 dataset [11] holds a 68% of attack records, but it's aggregated figure across 9 different attacks. This imbalance directly impacts model performance in two critical ways. First, models tend to become biased toward the majority (normal) class, achieving high overall accuracy but failing to detect rare attack instances, like fuzzing. Second, imbalance leads to the false positive problem: a model that is not properly tuned may classify normal traffic as malicious (false positive) or, worse, miss actual attacks (false negative).

To address these historical limitations, more comprehensive alternatives such as UNSW-NB15, CICIDS2017, and Kitsune [11, 12, 10] have emerged as the new standard, specifically incorporating fuzzing traffic within their captures. Beyond the mere presence of these attacks, the selection of modern datasets is driven by feature richness and realism; for instance, Kitsune's 115 statistical dimensions allow for fine-grained detection but require careful feature selection to avoid overfitting and high computational costs. Furthermore, these modern datasets prioritize generalizability by using live network traffic and preserving the temporal structure of packets, which is essential for capturing time dependencies in sequential models like LSTM or GRU. To ensure the reliability of the results, the literature emphasizes the importance of thorough preprocessing, including categorical encoding and min-max normalization, alongside standardized validation methods, such as 70/30 data splitting or 10-fold cross-validation, to confirm that the models are robust and not merely overfitted to the training data.

Table 2. Summary of Network Intrusion Detection Datasets Relevant to Fuzzing Attacks

Dataset	**Year**	**Attack Types**	**Fuzzing?**	**Records**	**Features**
KDD Cup 1999	1999	DoS, Probe, R2L, U2R	No	4.9M	41
UNSW-NB15	2015	9 categories (incl. Fuzzers)	Yes	257k	49
CICIDS2017	2017	7 categories	Partial	2.8M	80+
Kitsune	2018	Fuzzing (web API)	Yes	2.24M	115
CAN Bus	2021	DoS, Fuzzing, Spoofing, Replay	Yes	3.67M	5

Note: The CAN Bus dataset refers to the Car Hacking: Attack & Defense Challenge 2020 [8].

5. Discussion

While current research trends predominantly focus on building holistic Network Intrusion Detection Systems capable of identifying malicious traffic as a broad category, there is significant value in this generalized approach as a first line of defense. However, as cyber threats become more sophisticated, a shift toward threat-specific modeling is necessary. By isolating specific attack classes, such as network fuzzing, researchers can develop models that are fine-tuned to detect the subtle, protocol-specific anomalies that general classifiers might overlook. Emphasizing individual attack categories in future studies would not only improve detection accuracy but also enable more effective incident response strategies, as security agents would be equipped with models trained for the unique signatures of evolving attack techniques.

It is important to acknowledge that this comparative analysis was constrained by the relative lack of academic literature that isolates per-class performance metrics. During the research process, a recurring observation was that most studies prioritize aggregated accuracy figures over detailed evaluations of specific categories like fuzzing. This lack of transparency regarding individual class performance makes it challenging to perform a comprehensive analysis across a wider range of algorithms. Consequently, the results synthesized in this paper are based on a targeted selection of studies, highlighting a clear gap in the current research landscape that future publications should aim to fill by providing more transparent, per-class evaluation matrices.

6. Conclusions and future work

This report examined the performance of machine learning classification algorithms for detecting network fuzzing traffic, synthesizing results from recent academic studies across multiple datasets and experimental configurations. Among all evaluated algorithms, XGBoost distinguished itself as the most effective solution, achieving perfect precision, recall, and F1-score (1.00 each) with an overall accuracy of 0.985, while requiring only 6.4 seconds of training time, therefore resulting in a combination of classification performance and computational efficiency that no other evaluated method matched.

More broadly, the results suggest that machine learning models trained for fuzzing detection have matured considerably over the past decade, with modern approaches achieving near-perfect detection rates under controlled experimental conditions. However, this progress comes with an important aspect: the fuzzing techniques used to generate the datasets analyzed in this report reflect the attack tools available at the time of their creation. As offensive security research advances, new generations of fuzzing tools are increasingly incorporating adaptive and AI-driven strategies, capable of generating traffic that more closely mimics legitimate protocol behavior. Classical supervised classifiers, trained on static labeled datasets, may struggle to generalize against these evolving threats, suggesting that the current landscape of fuzzing detection, while promising, still requires deeper investigation to remain effective against future attack techniques.

Looking ahead, two directions stand out as particularly relevant for advancing this field. First, the exploration of unsupervised learning approaches, such as autoencoders or clustering-based anomaly detection. These approaches represent a promising avenue for detecting zero-day fuzzing attacks that do not match any patterns present in existing labeled datasets. Unlike supervised models, unsupervised methods do not rely on predefined attack signatures, making them inherently better suited to identifying novel and previously unseen threats. Second, the development of more comprehensive and publicly available datasets that include diverse, well-balanced representations of fuzzing traffic across different network environments remains a pressing need. As observed throughout this analysis, the scarcity of datasets with detailed per-class labeling is one of the main factors limiting the depth and comparability of research in this area. Addressing these two challenges would lay a stronger foundation for the next generation of threat-specific intrusion detection systems.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used Google's Gemini LLM in order to improve the language and readability of this paper. After using this tool/service, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

References

[1] Abuzir, Y., Abu Khalil, D., 2025. Improving network security through fuzzing attack detection: Machine learning on the kitsune dataset. Computer Science and Information Technology 13, 8–31.

[2] Alabdulatif, A., Rizvi, S.S.H., 2022. Machine learning approach for improvement in kitsune nid. Intelligent Automation & Soft Computing 32.

[3] Alalwany, E., Mahgoub, I., Alsharif, B., Alfahaid, A., 2025. An intelligent ensemble-based detection of in-vehicle network intrusion. Applied Sciences 15, 6869.

[4] Denning, D.E., 1987. An intrusion-detection model. IEEE Transactions on software engineering , 222–232.

[5] Dua, M., et al., 2019. Machine learning approach to ids: A comprehensive review, in: 2019 3rd International conference on Electronics, Communication and Aerospace Technology (ICECA), IEEE. pp. 117–121.

[6] Ghosh, A.K., Schwartzbard, A., 1999. A study in using neural networks for anomaly and misuse detection, in: 8th USENIX Security Symposium (USENIX Security 99).

[7] Giudici, P., Gramegna, A., Raffinetti, E., 2023. Machine learning classification model comparison. Socio-Economic Planning Sciences 87, 101560. URL: https://www.sciencedirect.com/science/article/pii/S0038012123000605, doi:https://doi.org/10.1016/j.seps.2023.101560.

[8] Kang, H., Kwak, B., Lee, Y., Lee, H., Lee, H., Kim, H., . Car hacking: Attack & defense challenge 2020 dataset. ieee dataport (2021).

[9] Li, J., Zhao, B., Zhang, C., 2018. Fuzzing: a survey. Cybersecurity 1, 6. URL: https://doi.org/10.1186/s42400-018-0002-y, doi:10.1186/s42400-018-0002-y.

[10] Mirsky, Y., Doitshman, T., Elovici, Y., Shabtai, A., 2018. Kitsune: An ensemble of autoencoders for online network intrusion detection, in: The Network and Distributed System Security Symposium (NDSS) 2018.

[11] Moustafa, N., Slay, J., 2015. Unsw-nb15: a comprehensive data set for network intrusion detection systems (unsw-nb15 network data set), in: 2015 Military Communications and Information Systems Conference (MilCIS), pp. 1–6. doi:10.1109/MilCIS.2015.7348942.

[12] Sharafaldin, I., Habibi Lashkari, A., Ghorbani, A.A., 2018. A detailed analysis of the cicids2017 data set, in: International conference on information systems security and privacy, Springer. pp. 172–188.

[13] Shushlevska, M., Efnusheva, D., Jakimovski, G., Todorov, Z., 2022. Anomaly detection with various machine learning classification techniques over unsw-nb15 dataset. URL: http://dx.doi.org/10.25673/76928.

[14] Tavallaee, M., Bagheri, E., Lu, W., Ghorbani, A.A., 2009. A detailed analysis of the kdd cup 99 data set, in: 2009 IEEE symposium on computational intelligence for security and defense applications, Ieee. pp. 1–6.

[15] Thapa, S., Mailewa, A., 2020. The role of intrusion detection/prevention systems in modern computer networks: A review, in: Conference: midwest instruction and computing symposium (MICS), pp. 1–14.